Hard-Coding prompts.py in Python

Line-by-line: grounded system rules, ChatPromptTemplate, and why RAG prompts look like this.

Project: logistics-rag-agent

File: prompts.py

Role: Keep the LLM honest -- answer only from retrieved context

This tiny module is the 'contract' between retrieved document chunks and the chat model. Without a grounded prompt, the model invents logistics policy from general knowledge.

1. Big picture

- GROUNDED_SYSTEM -- rules the model must follow
- GROUNDED_PROMPT -- a ChatPromptTemplate with {context} and {question} slots
- Used by rag.py: fill slots -> send to ChatOllama -> parse string answer

WHY

Separating prompts into their own file makes A/B testing wording easy without touching retrieval or agent code. Product and eng can edit text in one place.

2. Build step by step

Step 1: Module docstring + import

```
"""Grounded RAG prompts -- answer only from retrieved context."""

from langchain_core.prompts import ChatPromptTemplate
```

ChatPromptTemplate builds a chat-style prompt (system + human messages) with named variables you fill later via .invoke({'context': ..., 'question': ...}).

WHY

langchain_core is the stable core API. Prefer it over older langchain.prompts imports.

Step 2: Hard-code the system rules as a string constant

```
GROUNDED_SYSTEM = """You are a logistics domain assistant. Answer the user's question using ONL
Y the provided context from company SOPs and logistics documents.

Rules:
- Base every factual claim on the context below. Do not use outside knowledge.
- If the context is missing, incomplete, or unrelated to the question, reply exactly: I don't k
now
- Do not speculate, invent procedures, numbers, or sources.
- When you can answer, be concise and cite the document filename when helpful.
"""
```

What each rule buys you

- ONLY context -- blocks Wikipedia-style guessing about 'typical' SOPs
- Exact 'I don't know' -- makes evaluation and UI handling predictable
- No inventing numbers -- critical for logistics (wrong lead time = bad ops)
- Cite filename -- user can verify the source chunk

WHY

Grounding is a prompting strategy, not fine-tuning. Tight rules + temperature=0 (set in rag.py) are the usual way to keep answers close to PDFs.

Step 3: Wrap rules + user template in ChatPromptTemplate

```
GROUNDED_PROMPT = ChatPromptTemplate.from_messages(
    [
        ("system", GROUNDED_SYSTEM),
        (
            "human",
            """Context:\n{context}\n\nQuestion: {question}\n\nAnswer (or \"I don't know\" if th
e context is insufficient):""",
```

```
        ),
    ]
)
```

- ("system", ...) -- role message: identity + hard rules
- ("human", ...) -- user message with two placeholders: {context}, {question}
- {context} -- filled by rag.py with formatted retrieved chunks
- {question} -- the raw user question string

WHY

Putting Context above Question is deliberate: models attend strongly to nearby instructions; ending with 'Answer...' nudges a direct reply instead of a preamble.

WHY

from_messages([...]) is preferred over a single giant string because chat models are trained on role-tagged turns (system vs human).

3. Full file reference

```
"""Grounded RAG prompts -- answer only from retrieved context."""

from langchain_core.prompts import ChatPromptTemplate

GROUNDED_SYSTEM = """You are a logistics domain assistant. ..."""

GROUNDED_PROMPT = ChatPromptTemplate.from_messages([
    ("system", GROUNDED_SYSTEM),
    ("human", """Context:\n{context}\n\nQuestion: {question}\n\nAnswer (or \"I don't know\" if
the context is insufficient):"""),
])
```

4. How rag.py uses it

```
# inside build_rag_chain:
{'context': retriever|format, 'question': RunnablePassthrough()}
| GROUNDED_PROMPT
| llm
| StrOutputParser()
```

The dict before GROUNDED_PROMPT supplies the two template variables. The prompt object turns them into chat messages; the LLM returns an AIMessage; StrOutputParser extracts .content as a plain str.

5. Practice

- Change the fail phrase to 'Insufficient context.' and see agent/rag behavior change
- Remove 'ONLY the provided context' and ask a question not in the PDFs -- watch hallucination
- Add 'Always quote one short phrase from Context' and re-run ask()

6. Common mistakes

- Forgetting {context} or {question} names -- KeyError or empty fill at invoke time
- Putting facts in the system prompt instead of retrieval -- goes stale vs PDFs
- Vague 'be helpful' system text -- model fills gaps with world knowledge